Operations perform computations only with constant SCALAR values computable at assembly time.

Priority		
7	-	two's complement
7	+	no effect
7	~	one's complement
7	!	logic negation
4	<<	1<<4=16
4	>>	0xFE >> 0x0F
3	&	AND
2	^	XOR
1	|	1|2 = 3

5|6 + 7&8 = 5|(6+7)&8 = 5|13&8 = 5|8 = 13 = 0Dh !!

Any operation is on 64 bits

& - bitwise AND operator

AND - instr.

x AND 0 = 0

x AND 1 = x

x AND x = x

x AND ~x = 0

The & is useful for forcing some value of all bits to 0.

CLC

operator		
OR - instr.	X OR 0 = X	X OR X = X
	X OR 1 = 1	X OR ~X = 1

STC

The 1 is useful for forcing some value of some bits to 1.

operator.		
XOR - instr	X XOR 0 = X	X XOR X = 0
	X XOR 1 = ~X	X XOR ~X = 1

CMC

This is useful for negating the value of some bits.

CLC clear.
STC set
CMC - complent carry flag

Preset is diff methods for initializing with 0 the content of a register

AND EAX, 0
MOV EAX, 0
XOR EAX, EAX

a d? 17, -1, 'a'

b d? 200, -5, 'xy'

mov eax, ![a] ⇒ syntax error.

mov ecx, [!a] ⇒ not a declr. not determinable at assembly time.

mov eax, !a ⇐ not a valid operation

mov ecx, !(a+7) ⇐ syntax error.

mov eax, !(b-a)

↳ returns a scalar ⇒ OK

mov eax, !b-a

mov eax, !7 √ scalar = 0

!0 = 1

mov eax, !ebx

ac cgu 2

mov ah, !ac ⇐ constant (scalar).

mov ah, ~7 000...0111 ~ 111...1000 = F8h = -8

mov eax, ~7 FF FF FF F8h = -8

Do they represent the same value?

Depends on the interpretation (signed or unsigned)

~7 = -8

mov ah, 17 ^ ~17 = 1 = 0FFh

x XOR !x = 1.

mov ax, val ^ ~val

0ffffh

TASM - turbo assembler.

MASM - microsoft assembler

NASM - netwide assembler

Type operators and operands data types

V db ....

a dw ....

b dd .... direfferencing.

in NASM there are only addresses.

push v ← stack ← offset (v)

push [v] ← syntax error. (specify push dword [v] either word or dwordword.

operation size not specified.

INSTR. TYPE OPERATOR EXPR.

mov eax, [v] no syntax error. just take 4 bytes from v.

No data type associated to v.
It's just an address.

push 15 => translate to push dword 15

pop [v] (operation size not specified)

pop v. (v = constant) => syntax error.
v is not an L-VALUE

pop [eax] => syntax error (op. size not specified)

pop 15 => syntax error 15 not an L value.

pop [15] => op. size not specified.

pop dword [15] invalid memory specification

⇓

POP DWORD PTR DS:15.

mov [v], 0 => op. size not specified.

↓

byte, word, dword.

mov byte [v], 0 } the same.
mov [v], byte 0

div [v] ; op. size not specified.

imul [v+2] ; op. size not. specified.

mov a, b - syntax error.
mov [a], b - op. size not. specified.
word/dword.
mov a, [b] - a cannot be an L value.

mov [a], [b] - we cannot have
both explicit operands from memory.

mul ax - mul reg/mem.
mul, [v] op. size not specified.
byte/word/dword.

mul eax ✓
mul, [eax] op. size not. specified.
byte, word, dword.

mul 15 - invalid op. code.

pop byte [v] - invalid. combination of op-code (either w or sw)

pop qword [rest] - not supported in 32 bit mode.

